MODULE 17 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

Agents I: design

Agent types, roles and goals, instructions versus controls, tool schemas, tool_choice, and conversation tracking.

DOMAIN 2 - Generative AI and agents WEIGHT 30-35% MODULE 17 of 39

admin@sthithapragna.com

An agent is a new intern. Instructions are the **job description**. Tools are the **doors**. The **keycard** decides which doors open.

admin@sthithapragna.com

Model, instructions, tools

- Every agent combines a **model**, **instructions** and **tools**.
- An agent is a persisted, **named and versioned** asset in the Foundry project.
- Agents are identified by **name and version**. The old agent ID GUID no longer exists.
- You run an agent by creating a **response** that references it.
- Treat instructions and conversations as application data: **no secrets** in either.

> ON THE EXAM
>
> Older samples use agent IDs, threads and runs. That's Agents (classic), which retires on 31 March 2027.

Kinds of agent in Foundry Agent Service

TYPE	WHAT IT IS	CHOOSE IT WHEN
Prompt agent	Configuration only: model, instructions, tools	No custom code needed; fastest to build
Hosted agent (preview)	Your own code in a container, run by Foundry	Your framework, custom protocols, compute control
Workflow agent (preview)	Visual declarative workflows	Existing ones only: retiring 1 December 2026
Agents (classic)	Threads and runs	Migrate away: retiring 31 March 2027

ON THE EXAM

After 1 December 2026, workflow YAML still runs when deployed as a hosted agent. New workflows belong in Agent Framework.

No custom code needed? A **prompt agent**.
Your own framework, protocol or compute?
A **hosted agent**.

admin@sthithapragna.com

Writing the job description

- **Role:** who the agent is, and who it serves. 'A returns assistant for retail customers.'
- **Goal:** what done looks like. 'The customer has a return label, or a clear reason why not.'
- **Boundaries:** what it declines, and when it hands over to a person.
- **Tool guidance:** what each tool is for, when to use it, and what to do if it fails.
- **Output contract:** the format and tone of replies, especially if code reads them.

> ON THE EXAM
>
> Microsoft's tool guidance: say what to do when a tool fails or returns nothing, so the agent explains rather than invents.

Instructions steer. Controls enforce.

INSTRUCTIONS

The job description

- Shape behaviour and tone
- Usually followed, never guaranteed
- Can be overridden by injection or error

CONTROLS

The keycard

- Validation inside the tool's own code
- Allowed tools and least-privilege access
- Approval before consequential actions
- Roles on the systems the tools reach

THE TIEBREAKER

Every 'must never' in the instructions needs a matching control outside the prompt.

A prompt agent with a function tool

```
order_tool = FunctionTool(
    name="get_order_status",
    description="Look up delivery status for one order. Use when the "
                "customer asks where their order or parcel is.",
    parameters={"type": "object",
        "properties": {"order_id": {"type": "string",
            "description": "Order number, for example ORD-10442"}},
        "required": ["order_id"], "additionalProperties": False},
    strict=True)
agent = project.agents.create_version(agent_name="returns-assistant",
    definition=PromptAgentDefinition(model="chat-prod",
        instructions=returns_instructions, tools=[order_tool]))
```

ON THE EXAM

The description tells the model when to call the tool. The schema tells it exactly what to send.

Designing tools the model uses well

- **Specific names:** get_order_status, not api_call or helper.
- **Descriptions say what and when**, including which requests should trigger the tool.
- **Parameter descriptions with examples**, such as the expected ID format.
- **Strict schemas:** required fields, no additional properties, strict set to true.
- **Small, focused tools** beat one 'run any query' tool, which is hard to use and hard to secure.

> ON THE EXAM
>
> Strict schemas guarantee the shape of arguments, not their truth. Validate values in the tool's code.

Getting the right tool called

- Say in the instructions **what each tool is for**, and when to use it.
- Where tools overlap, add a **decision rule**: 'Use file search before web search for internal content.'
- **tool_choice** gives deterministic control: **auto**, **required** or **none**.
- A tool needs support from **both the model and the region**, or it won't run.
- Check traces to see whether a tool call was even attempted.

> THE SIGNAL
>
> 'The agent never calls the tool' checks, in order: attached, supported, described clearly, then tool_choice.

How function calls flow in an agent

- The agent returns a **function call** with arguments. **Your application runs** the function.
- You return the result as a function call output, matched by its **call ID**.
- Submit tool outputs within **10 minutes**, before the run expires.
- The portal can run agents with function tools, but **can't add or edit** function definitions.
- Define and update function tools with the **SDK or REST API**.

> ON THE EXAM
>
> Function tools run in your code, so your code is where the keycard checks belong.

Four ways to track a conversation

APPROACH	HOW IT WORKS	CHOOSE IT WHEN
`Conversation object`	Durable ID stores every item	Users return, or switch channels
`previous_response_id`	Chain to the last stored response	Simple multi-turn, one session
`store=false`	Your app resends earlier items	Nothing may be stored
`Memory (preview)`	Facts and summaries across sessions	Preferences, not transcripts

> ON THE EXAM
>
> The exam expects you to choose one of these deliberately.

How conversation objects work

- A conversation is a **durable object with an ID**, reusable across sessions and channels.
- It stores **items**: messages, tool calls, tool outputs and responses.
- Each response gets the full conversation as input, and appends its output to it.
- If history exceeds the context window, the **input** is truncated. The conversation isn't.
- Conversations replace the classic **thread** pattern.

> ON THE EXAM
>
> Long conversations silently lose early turns from the model's view. Summaries or memory keep key facts in reach.

Two turns in one conversation

```
openai = project.get_openai_client()
conversation = openai.conversations.create()

agent_ref = {"agent_reference": {"name": "returns-assistant",
                                 "type": "agent_reference"}}

first = openai.responses.create(conversation=conversation.id,
    extra_body=agent_ref, input="Where is order ORD-10442?")

follow_up = openai.responses.create(conversation=conversation.id,
    extra_body=agent_ref, input="Can I return it instead?")
```

ON THE EXAM

The same conversation ID carries context. Save it with the customer's case to continue on another day or device.

Conversation history or memory?

CONVERSATION

The transcript

- Every message and tool call in one dialogue
- Continues that dialogue later
- Grows with every turn

MEMORY (PREVIEW)

The notes

- Extracted facts: 'allergic to dairy'
- Summaries carried into new conversations
- Scoped per user

THE TIEBREAKER

Continue the same case: conversation. Recognise the customer in a brand-new case: memory.

An agent design checklist

DECISION	GOOD ANSWER LOOKS LIKE
Agent type	Prompt unless custom code is needed
Instructions	Role, goal, boundaries, tool guidance, output contract
Tools	Few, focused, clearly described, strict schemas
Controls	Every 'must never' enforced outside the prompt
Conversation tracking	Chosen deliberately for sessions and retention
Operations	Tracing on, evaluated before release

ON THE EXAM

Modules 18 to 20 build on each row: tools and memory, multiple agents, then autonomy with safeguards.

> An HR team wants an internal agent that answers policy questions using file search over its handbook. It needs no custom logic.
>
> They want it working this week, with nothing to host, patch or scale themselves.

What should the team build?

A. A prompt agent with instructions, a model and the file search tool.

B. A hosted agent packaged as a container and pushed to Container Registry.

C. A workflow agent designed in the Foundry portal's visual builder.

D. A classic agent using threads and runs, following an older tutorial.

◆ PAUSE AND DECIDE

What should the team build?

A Prompt agent with file search.

Configuration only, built-in tool, and Foundry hosts it. Nothing to maintain.

B Hosted agent container.

For custom code. Adds a container to build and maintain for no benefit here.

C Workflow agent.

The visual workflow builder retires on 1 December 2026.

D Classic threads and runs.

Agents (classic) retire on 31 March 2027. Don't start new work there.

THE SIGNAL

'No custom logic' plus 'nothing to host' is the definition of a prompt agent.

> A telecom support agent handles billing disputes that often take several days. Customers start on the website, then return on the mobile app.
>
> Today the web client keeps history in browser memory, so every return starts from scratch and the agent repeats questions.

How should the team track conversations?

A Store the latest previous_response_id in each browser's local storage.

B Set store to false and resend history held in the browser each turn.

C Deploy a model with a larger context window so more history fits.

D Create a conversation per case, and save its ID with the case record.

◆ PAUSE AND DECIDE

How should the team track conversations?

A previous_response_id in local storage.

Tied to one browser. The mobile app can't see it.

B store=false, browser history.

History still lives on one device, which is the current problem.

C Larger context window.

Fits more history, but doesn't give the history a home across devices.

D Conversation per case, ID with the case.

A durable server-side conversation any channel can continue, days later.

THE SIGNAL

'Several days' plus 'switches channel' means state must live server-side, keyed to something both apps can find.

> A retail agent has a function tool named api_call with the description 'Order API'. Its only parameter is a string called q.
>
> When customers ask 'where's my parcel?', the agent usually answers vaguely from general knowledge instead of calling the tool.

What should the team do first?

A Raise the temperature so the agent tries a wider range of actions.

B Edit the function definition in the portal to force it to be used.

C Name and describe the tool and parameter clearly, with when to use it.

D Switch the agent to a larger reasoning model with high reasoning effort.

◆ PAUSE AND DECIDE

What should the team do first?

A Raise temperature.
Adds randomness. It doesn't explain what the tool is for.

B Edit the function in the portal.
The portal can't add or edit function definitions. Use the SDK or REST API.

C Clear name, descriptions, and when to use it.
The model can't connect 'parcel' to 'Order API' with a parameter called q.

D Larger reasoning model.
Costlier, and still guessing what a vague tool does.

THE SIGNAL

Microsoft's fix for tools not being called: better descriptions and instructions first, then tool_choice if a call must happen.

> A shop's agent instructions say 'Never issue refunds over 100 pounds without a manager'. Its issue_refund tool accepts any amount.
>
> A customer's message persuaded the agent to issue a 900-pound refund, which went through immediately.

What should the team change?

A Repeat the refund rule in capitals at the start and end of the instructions.

B Enforce the limit in the refund tool's code, and require approval above it.

C Set tool_choice to none so the agent can never call any of its tools.

D Add a violence guardrail at the user input intervention point, set to block.

◆ PAUSE AND DECIDE

What should the team change?

A Stronger instructions.

Still a job description. The next persuasive message can talk past it.

B Limit in tool code, approval above it.

The tool itself refuses large refunds unless a manager approves. A keycard, not a request.

C tool_choice none.

Stops every tool, breaking the agent's legitimate work.

D Violence guardrail.

A persuasive refund request isn't violent. The classifier won't see it.

THE TIEBREAKER

A 'never' in the instructions, with a tool that allows it anyway, is a missing keycard.

Agent design in five lines

- **Agent = model + instructions + tools**, named and versioned. No more agent ID GUIDs.
- **Prompt agent** unless you need your own code; then a **hosted agent**.
- **Tools:** specific names, what-and-when descriptions, strict schemas, tool_choice when needed.
- **Every 'must never' needs a control outside the prompt.**
- **Conversation** to continue a case; **memory** to recognise a person.

ON THE EXAM

Next, module 18: agent tools and memory, including MCP, OpenAPI, search, file search and code interpreter.